

Technology Skill Development Lab Java

Assignment No.4

Student Name : Saurabh Ashok Wagh

Roll No. : 26

Division : A

Batch : A2

Class : SY

Branch : B.Tech AIDS

Course Teacher Name : Prof. Vishakha Salunkhe

Problem Statement : Multiple inheritance using Interface:

a) Define two interfaces: CardioWorkout and StrengthTraining. In the CardioWorkout interface, declare methods such as getWorkoutName(), getDurationMinutes(), and displayWorkoutDetails().

b) In the StrengthTraining interface, declare methods like getEquipmentUsed(), getInstructorName(), and displayStrengthTrainingDetails().

c) Derived Class: Create a class named FullBodyWorkout that implements both Cardio_Workout and StrengthTraining. This class represents a workout program that combines cardio exercises with strength training for a comprehensive full-body workout. Design a Java program using interfaces to implement a simple AI-based recommendation system. Create interfaces such as UserPreference and RecommendationEngine. The program should store product or course data and recommend suitable items to users based on predefined rules such as price range, category, or rating.

Program:

```
interface CardioWorkout {
    String getWorkoutName();
    int getDurationMinutes();

    void displayWorkoutDetails();
}

interface StrengthTraining {
    String getEquipmentUsed();
    String getInstructorName();

    void displayStrengthTrainingDetails();
}
```

```
class FullBodyWorkout implements CardioWorkout, StrengthTraining {

    private String workoutName;
    private int durationMin;
    private String equipmentUsed;
    private String instructorName;

    FullBodyWorkout(String workoutName, int durationMin,
                    String equipmentUsed, String instructorName) {

        this.workoutName = workoutName;
        this.durationMin = durationMin;
        this.equipmentUsed = equipmentUsed;
        this.instructorName = instructorName;
    }

    // Implementing CardioWorkout methods
    public String getWorkoutName() {
        return workoutName;
    }

    public int getDurationMinutes() {
        return durationMin;
    }

    public void displayWorkoutDetails() {
        System.out.println("Workout Name is: " + getWorkoutName());
```

```
        System.out.println("Workout Duration is: " + getDurationMinutes() + " minutes");
    }

    // Implementing StrengthTraining methods
    public String getEquipmentUsed() {
        return equipmentUsed;
    }

    public String getInstructorName() {
        return instructorName;
    }

    public void displayStrengthTrainingDetails() {
        System.out.println("Equipment used in a workout is: " + getEquipmentUsed());
        System.out.println("Instructor name is: " + getInstructorName());
    }
}

public class Main {
    public static void main(String[] args) {

        FullBodyWorkout f1 = new FullBodyWorkout("Bicep", 10, "Dumbbell", "ABC");

        f1.displayWorkoutDetails();
        f1.displayStrengthTrainingDetails();
    }
}
```

Output :

```
Workout Name is: Bicep
Workout Duration is: 10 minutes
Equipment used in a workout is: Dumbbell
Instructor name is: ABC
```

Conclusion: The program successfully demonstrates multiple inheritance using interfaces in Java. The FullBodyWorkout class implements both CardioWorkout and StrengthTraining interfaces and provides implementations for all their methods. It displays the workout details as well as strength training details. Thus, the program shows how interfaces allow a class to implement multiple behaviors in Java.

Problem Statement : Design a Java program using interfaces to implement a simple AI-based recommendation system. Create interfaces such as UserPreference and RecommendationEngine. The program should store product or course data and recommend suitable items to users based on predefined rules such as price range, category, or rating.

Program:

```
interface UserPreference {
    String getCategory();
    double getMaxPrice();
    double getMinRating();
}

interface RecommendationEngine {
    void recommend();
}

class Product {
    String name;
```

```
    String category;
    double price;
    double rating;
}

class AIRecommendation implements UserPreference, RecommendationEngine {

    String category = "Programming";
    double maxPrice = 50;
    double minRating = 4.5;

    public String getCategory() {
        return category;
    }

    public double getMaxPrice() {
        return maxPrice;
    }

    public double getMinRating() {
        return minRating;
    }

    public void recommend() {

        Product p1 = new Product();
        p1.name = "Java Course";
```

```
p1.category = "Programming";
p1.price = 50;
p1.rating = 4.8;

Product p2 = new Product();
p2.name = "Python Course";
p2.category = "Programming";
p2.price = 40;
p2.rating = 4.6;

Product p3 = new Product();
p3.name = "Marketing Course";
p3.category = "Marketing";
p3.price = 30;
p3.rating = 4.7;

System.out.println("Recommended Products:");

if (p1.category.equals(category) &&
    p1.price <= maxPrice &&
    p1.rating >= minRating)
    System.out.println(p1.name);

if (p2.category.equals(category) &&
    p2.price <= maxPrice &&
    p2.rating >= minRating)
    System.out.println(p2.name);
```

```
        if (p3.category.equals(category) &&
            p3.price <= maxPrice &&
            p3.rating >= minRating)
            System.out.println(p3.name);
    }
}

public class Main {
    public static void main(String[] args) {

        AIRecommendation user = new AIRecommendation();

        user.recommend();
    }
}
```

Output :

```
Recommended Products:
Java Course
Python Course
```

Conclusion: The program successfully implements a simple AI-based recommendation system using interfaces. It uses UserPreference to store the user's preferred category, maximum price, and minimum rating, while RecommendationEngine provides the recommendation method. The system recommends products that satisfy all the predefined conditions. This demonstrates how interfaces can be used to create a simple and flexible recommendation system in Java.